
RAPPORT FIN D'ETUDE : SYNCHRONISATION DE CAMERAS IP

AUGUST 23, 2022

ÉTS

Alexandre BLANC

Sommaire :

Sommaire :	1
Contexte	3
Définition :	3
Méthode possible de calibration temporelle	4
Solution matérielle	4
Solution 1 : Firewire	4
Solution 2 : NTP	4
Solution logicielle	4
Solution 1 : synchronisation par extraction d'un plan	4
Solution 2 : synchronisation par étude des trajectoires d'objets en mouvement	4
Solution 3 : synchronisation par étude des changements de luminosité	5
Solution 4 : SMPTE (timecode)	5
ST-2059	6
Variable	6
General	6
Vidéo	6
Audio	6
Timecode	7
SMPTE Epoch	8
Alignement des signaux	8
Génération de signal vidéo	8
Analog Standard Definition Television	8
Digital Standard Definition Television Interfaces	9
Analog High-Definition Television	9
Digital High Definition and Ultra High-Definition Television Interfaces	9
Génération de la valeur d'adresse de temps SMPTE ST 12-1	10
Étape 1 :	11
Étape 2 :	12
Étape 3 :	12
Étape 4 :	13

Étape 5 :	14
Étape 6 :	14
Fonctionnement site web caméra Ottica 4k.....	15
Fonctionnement API Insta360 :.....	18
Solution retenue	21
Explication et évolution de la solution.....	21
Etape 1 : Lancement des deux Stream.....	21
Etape 2 : Code de synchronisation.....	22
1/ Méthode	24
2/Matériel	24
3/ Déroulement et explication d'une synchronisation	25
1) Connection aux caméras	25
2) Lancement des threads	28
3) Initialisation des timecodes	29
4) Détection de variation de lumière et activation des timecodes	29
5) Synchronisation temporelle et affichage des streams	34
6) Vérification de la synchronisation	38
7) Fermeture des streams	40
4/ Conclusion.....	40
Piste d'évolution :	41
8) Annexe	42
Code	43

Contexte

Les quatre caméras sont connectées au réseau (LAN) et on aura besoin de les synchroniser en temps pour capturer la scène en multi vue. Nous avons 3 Insta360 et une Ottica qu'on aimerait synchroniser en temps. La tâche est de créer un outil logiciel pour assurer la synchronisation en temps de l'acquisition et l'intégrer avec l'outil de calibration spatiale.

Il faudra explorer les API fournies par les caméras pour trouver une façon de faire cette synchronisation. Idéalement, on aimerait utiliser le standard de synchronisation de transmission vidéo SMPTE. Ceci dépend des capacités des caméras et de l'équipement réseau, alors on n'a pas de garantie que c'est possible.

Definition :

SMPTE : C'est un ensemble de normes coopérant pour marquer des trames individuelles de vidéo ou d'un film avec un code temporel.

Timecode : Un timecode, ou code temporel, est une référence temporelle utilisée dans les domaines du son et de l'image, pour la synchronisation et le marquage de matériaux enregistrés. Le timecode (TC) s'exprime en heures, minutes, secondes et images

ST-2059 : SMPTE 2059 est une norme qui décrit comment synchroniser l'équipement vidéo sur un réseau IP. La norme est basée sur IEEE 1588-2008.

- SMPTE 2059-1 – Définit la génération de signal basée sur les informations temporelles fournies par le protocole IEEE 1588.
- SMPTE 2059-2 – Définit un profil de fonctionnement pour le protocole IEEE optimisé pour les besoins de la synchronisation multimédia.

PTP : c'est un protocole Ethernet de synchronisation d'horloge, normalisé sous le nom IEEE 1588. Ce protocole est destiné aux réseaux supportant le multicast et l'unicast.

Méthode possible de calibration temporelle

La synchronisation de flux vidéo peut être fait de façon matérielle ou logiciels. La synchronisation matérielle s'appuie sur l'utilisation de circuits électroniques dédiés. La synchronisation logicielle, elle se base plutôt sur une analyse du contenu des images.

Solution matérielle

Solution 1 : Firewire

Utilisation d'une connexion par port possédant une interface multiplexée suivant la norme **IEEE 1394**, aussi appelé FireWire. Des caméras reliées entre elles par un bus de données via leur port FireWire peuvent être synchronisées. Mais, le nombre de caméras connecté est limité par la capacité de débit du bus. De plus tous les types de caméras ne peuvent pas utiliser cette méthode. En effet ; cette méthode est peu souple sur des équipements vidéo hétéroclites.

Solution 2 : NTP

Cette solution consiste à envoyer via ordinateurs des impulsions de synchronisations à destination des caméras, ou chaque caméra est reliée à un ordinateur. Pour ce faire, le but est de synchroniser chaque ordinateur entre eux. Soit on utilise le port parallèle de chaque ordinateur pour les relier. Soit on passe par un réseau, utilisant le protocole Ethernet. Dans ce cas-là, un ordinateur maître envoie des impulsions de synchronisation aux autres ordinateurs esclaves, l'ordinateur maître étant par exemple un serveur de réseau utilisant un protocole NTP (Network Time Processing). Ce protocole permet de synchroniser des horloges de système à travers un réseau de transmission de paquets dont la latence est variable.

Solution logicielle

Solution 1 : synchronisation par extraction d'un plan

Cette première méthode détermine l'équation d'un plan formé par les trajectoires de tous les objets en mouvement dans la scène. Ce plan permet de relier toutes les caméras entre elles. Il s'agit ensuite de trouver une projection homographique dans le plan, des trajectoires obtenues par les différentes caméras, telle que l'erreur de projection homographique soit minimale. En effet, l'erreur de projection est minimale pour des points synchrones de trajectoire se correspondant dans deux flux vidéo. Un inconvénient de cette méthode est qu'il n'est pas toujours possible de trouver une projection homographique satisfaisant le critère de minimisation de l'erreur de projection.

Solution 2 : synchronisation par étude des trajectoires d'objets en mouvement

Cette méthode repose sur les trajectoires d'objets dans un couple de séquences vidéo désynchronisées. Un algorithme de type RANSAC, Random Sample Consensus est notamment utilisé afin de sélectionner des paires de trajectoires candidates. Les trajectoires mises en correspondance permettent d'estimer une matrice fondamentale de correspondance de ces trajectoires. La qualité de la matrice fondamentale est d'autant plus correcte que les trajectoires mises en correspondance sont synchrones. La synchronisation est alors obtenue par un algorithme itératif sur la qualité de la matrice fondamentale. Le problème de cette méthode est que pour synchroniser les caméras, elles doivent observer l'ensemble de la trajectoire.

Solution 3 : synchronisation par étude des changements de luminosité

Cette solution procède à un alignement des séquences en fonction de leur variation de luminosité. Cette solution permet de s'affranchir de l'analyse d'objets en mouvement dans une scène qui peut par exemple en être dépourvue. Cependant les capteurs des caméras sont plus ou moins sensibles aux variations lumineuses. De plus l'orientation des caméras modifie également la perception des variations lumineuses. Cette quatrième solution est donc peu robuste lorsqu'elle est utilisée dans un milieu où la luminosité de la scène n'est pas maîtrisée. Cette troisième solution nécessite également un calibrage fin de la colorimétrie des caméras, ce qui n'est pas toujours possible avec des caméras miniaturisées basiques.

Solution 4 : SMPTE (timecode)

Cette solution de synchronisation se base sur le fait que le maître génère et distribue son timecode qui servira de référence aux timecodes des esclaves. Les esclaves n'ont pas obligatoirement le même timecode que le maître, dans ce cas-là, on leur attribue un *offset*. Le problème du timecode est que ce procédé ne peut assumer une synchronisation qu'à « l'image près ». Le code temporel SMPTE est présenté au format *heure:minute:seconde:image* et est généralement représenté sur 32 bits à l'aide de décimal codé binaire. En général, les informations de fréquence d'images du code temporel SMPTE sont implicites, connues à partir du taux d'arrivée du code temporel à partir du support. Il peut également être spécifié dans d'autres métadonnées encodées dans le support. Le code temporel longitudinal est généré par le générateur de synchronisation maître du studio et distribué à partir d'un point central. Les générateurs de synchronisation centrale dérivent généralement leur synchronisation d'une horloge atomique, en utilisant soit l'heure du réseau, soit le GPS.

Le code temporel peut être attaché à un support d'enregistrement de différentes manières :

- Timecode linéaire, LTC : adapté pour être enregistré sur un canal audio, ou transporté par des fils audio pour une distribution dans un studio pour synchroniser les enregistreurs et les caméras. Pour lire le LTC, l'enregistrement doit être en mouvement, ce qui signifie que le LTC est inutile lorsque l'enregistrement est stationnaire ou presque stationnaire.

- Timecode VITC : enregistré dans l'intervalle de suppression verticale du signal vidéo sur chaque image de la vidéo. L'avantage du VITC est que, puisqu'il fait partie de la vidéo de lecture, il peut être lu lorsque la bande est à l'arrêt.

- Timecode AES-EBU intégré : timecode SMPTE intégré dans une connexion audionumérique AES3.

- Code temporel longitudinal de la piste de contrôle, CTL : code temporel SMPTE intégré dans la piste de contrôle d'une bande vidéo.

- Code temporel visible, BITC : les chiffres sont gravés dans l'image vidéo afin que les humains puissent facilement lire le code temporel.

ST-2059

Cette partie explique le fonctionnement du protocole ST-2059, la librairie timecode, que nous utiliserons plus tard, a le même principe

Variable

General

T : le temps continu écoulé depuis le SMPTE Epoch en secondes.

Alignment Period : durée entre les points d'alignement

Alignment Time : durée à partir du SMPTE Epoch à laquelle un point d'alignement se produit

NextAlignmentTime : l'heure à laquelle le prochain point d'alignement sur le signal d'interface se produira, en fonction du temps courant, t.

SR : la fréquence de l'horloge d'échantillonnage en Hz

Vidéo

Cf : durée d'une séquence de champ de couleur dans les champs.

ColorField : valeur du compteur de champ de couleur (base un). La valeur 1 de ColorField est le 1er champ de la séquence de champ de couleur (telle que définie dans les normes de référence).

H : durée d'une ligne vidéo entière en périodes d'horloge d'échantillonnage.

HA : durée d'une ligne vidéo active en périodes d'horloge d'échantillonnage.

L : numéro de ligne du point d'alignement vertical.

LineNumber : valeur du compteur vertical (base un). LineNumber 1 est la ligne 1 du cadre.

P : numéro d'échantillon du point d'alignement horizontal.

R : fréquence d'images vidéo en images par second.

SampleWordNumber : valeur du compteur d'échantillons horizontaux. SampleWordNumber 0 est le 1er échantillon actif de la ligne.

Tenfield : valeur du compteur de dix champs (base un). La valeur Tenfield 1 correspond au début de l'identification de la séquence à dix champs.

V : taille d'un cadre complet en lignes.

Audio

FCnt : position de la trame dans un bloc AES3.

Tsamp : période de l'horloge d'échantillonnage.

tsCnt : position de l'intervalle de temps dans une sous-trame AES3.

TtsClk : période d'horloge de tranche de temps.

Timecode

BitNumber : numéro de base zéro du bit du mot de code LTC.

CF : durée d'une séquence de champ de couleur en images.

colorFrameIdentificationMode : indicateur qui spécifie si l'identification du cadre de couleur sur le SMPTE Epoch utilisée.

currentLocalOffset : décalage en secondes d'heure locale par rapport à l'heure PTP.

currentLocalOffsetActive : décalage actif en secondes de l'heure locale par rapport à l'heure PTP.

dayLightSavingMode : indicateur qui spécifie si l'heure d'été est en vigueur.

JJ : jour d'une date SMPTE ST 309 transportées dans une valeur d'adresse de temps.

dropFrameMode : indicateur qui spécifie si le drop frame ou non-frame est utilisé.

Ff : taux de mots de code LTC en mots de code par seconde.

FF : trames de la valeur de l'adresse temporelle.

FF_{pdjam} : images de la valeur de l'adresse temporelle lors de l'événement Daily Jam précédent.

HH : heures de la valeur de l'adresse de temps.

HH_{pdjam} : heures de la valeur de l'adresse de temps lors de l'événement Daily Jam précédent.

jumpSeconds : taille de la prochaine discontinuité, en seconde, de l'heure locale.

leapSecondJump : indicateur qui est vrai lorsque la prochaine discontinuité de l'heure locale sera due à l'insertion ou suppression d'une seconde intercalaire.

MJD : version abrégée de la méthode de datation de la date julienne (JD).

MM : minutes de la valeur de l'adresse de temps.

MM_{pdjam} : minutes de la valeur de l'adresse de temps lors de l'événement Daily Jam précédent.

Mo : mois d'une date SMPTE ST 309 transportée dans une valeur d'adresse de temps.

previousJamLocalOffset : décalage en secondes de l'heure locale par rapport à l'heure PTP qui était active au précédent jam quotidien. Il s'agit du champ "previousJamLocalOffset" dans le profil SMPTE.

previousDayLightSavingMode : indicateur qui spécifie si l'heure d'été est en vigueur au jour précédent Événement Jam.

SS : secondes de la valeur de l'adresse de temps.

SS_{pdjam} : secondes de la valeur de l'adresse de temps lors de l'événement Daily Jam précédent.

t_{epdjamPTP} : heure exacte de l'événement Daily Jam précédent en temps PTP.

SMPTE Epoch

Le SMPTE Epoch est le 01 January 1970 00 :00 :00 TAI.

Alignement des signaux

Dans tous les cas où l'alignement des signaux est défini dans la présente norme, les temps d'alignement doivent être les suivants :

$$AlignmentTime = n \times AlignmentPeriod$$

Avec n entier

Si l'heure actuelle est tt , le prochain point d'alignement sur le signal d'interface se produira à :

$$NextAlignmentTime = \text{floor}\left(\frac{t}{AlignmentPeriod} + 1\right) \times AlignmentPeriod$$

Génération de signal vidéo

Voir annexe pour les tableaux génération de signal

Analog Standard Definition Television

La période entre les points d'alignement doit être :

$$AlignmentPeriod = \frac{1}{R}$$

Si la séquence de trames de couleur doit être alignée, la période entre les points d'alignement doit être :

$$AlignmentPeriod = \frac{Cf}{R \times 2}$$

Le compteur vertical doit être :

$$LineNumber = (\text{floor}(t \times R \times V + (L - 1)) \% V) + 1$$

Le compteur de champ de couleur doit être :

$$ColorField = \left(\text{floor}\left(2 \times \frac{t \times R \times V + (L - 1)}{V}\right) \% Cf \right) + 1$$

$$TenField = \left(\text{floor}\left(2 \times \frac{t \times R \times V + (L - 1)}{V}\right) \% 10 \right) + 1$$

Digital Standard Definition Television Interfaces

La période entre les points d'alignement doit être :

$$AlignmentPeriod = \frac{(H \times V)}{SR} = \frac{1}{R}$$

Les compteurs horizontaux et verticaux doivent être :

$$SampleWordNumber = (\text{floor}(t \times SR) + P) \% H$$

$$LineNumber = \left(\left(\text{floor} \left(\frac{((t \times SR) + P - HA)}{H} \right) + (L - 1) \right) \% V \right) + 1$$

Analog High-Definition Television

La période entre les points d'alignement doit être :

$$AlignmentPeriod = \frac{(H \times V)}{SR} = \frac{1}{R}$$

Digital High Definition and Ultra High-Definition Television Interfaces

La période entre les points d'alignement doit être :

$$AlignmentPeriod = \frac{(H \times V)}{SR} = \frac{1}{R}$$

Les compteurs horizontaux et verticaux doivent être :

$$SampleWordNumber = (\text{floor}(t \times SR) + P) \% H$$

$$LineNumber = \left(\left(\text{floor} \left(\frac{((t \times SR) + P - HA)}{H} \right) + (L - 1) \right) \% V \right) + 1$$

Génération de la valeur d'adresse de temps SMPTE ST 12-1

La charge utile de la valeur de l'adresse temporelle SMPTE ST 12-1 doit être calculée à l'aide des étapes suivantes :

Étape 1 : (Démarrage) Calculez la date exacte, l'heure et la valeur de l'adresse de temps de l'événement Daily Jam précédent.

Étape 2 : (Démarrage) Déterminer l'heure du premier mot de code à générer.

Étape 3 : (Tester le Daily Jam) Vérifiez si l'événement Daily Jam s'est produit.

Étape 4 : (Mettre à jour les valeurs quotidiennes de bourrage) Si un événement de bourrage quotidien s'est produit, calculez la date, l'heure et Heure Valeur de l'adresse de l'événement Daily Jam local.

Étape 5 : (Sortie) Calculer la valeur de l'adresse horaire SMPTE ST 12-1 et la date SMPTE ST 309 en utilisant la date, l'heure et l'adresse exactes de l'événement Daily Jam précédent.

Étape 6 : (Continuer) Incrémenter le mot de passe ; revenez à l'étape 3.

Étape 1 :

1. Calculer l'heure du Daily Jam précédent en temps PTP

$$t_{pdjamPTP} = timeOfPreviousJam$$

2. Calculez le nombre de mots de code ($n_{previousJam}$) depuis le SMPTE Epoch au ou juste après Jam time :

$$n_{previousJam} = \text{ceiling}(t_{pdjamPTP} \times Ff)$$

if Ff == 30/1.001 then

$$CF = 2$$

else if Ff == 25 then

$$CF = 4$$

endif

- a. Calculer le nombre de mots de code (n_{cf}) jusqu'au début de la séquence de champ de couleur

$$n_{cf} = (CF - (n_{previousJam} \% CF)) \% CF$$

- b. Calculer le nombre ajusté de mots de code jusqu'au début de la séquence de champ de couleur

$$n_{previousJam} = n_{previousJam} + n_{cf}$$

3. Calculer l'heure exacte de Daily Jam en temps PTP ajusté au début du mot de code suivant

$$t_{epdjamPTP} = \frac{n_{previousJam}}{Ff}$$

4. Calculer l'heure exacte de Daily Jam sur l'échelle de l'heure locale

$$t_{epdjamLocal} = t_{epdjamPTP} + previousJamLocalOffset$$

if (t_epdjamLocal < 0)

$$t_{epdjamLocal} = t_{epdjamLocal} + (24 \times 60 \times 60)$$

5. Définissez la valeur de l'adresse horaire sur l'heure du Daily Jam :

$$HH_{pdjam} = \text{floor}\left(\frac{t_{epdjamLocal}}{60 \times 60}\right) \% 24$$

$$MM_{pdjam} = \text{floor}\left(\frac{t_{epdjamLocal}}{60}\right) \% 60$$

$$SS_{pdjam} = 0$$

if(Ff == 25) *and* (colorFrameIdentificationMode == 1) *then*

$$FF_{pdjam} = 1$$

else

$$FF_{pdjam} = 0$$

endif

Étape 2 :

$$t_{codeword} = \frac{\text{ceiling}(t \times Ff)}{Ff}$$

Le numéro de mot de code correspondant (en partant de zéro au SMPTE Epoch) est :

$$\begin{aligned} n_{codeword} &= t_{codeword} \times Ff \\ &= \text{ceiling}(t \times Ff) \end{aligned}$$

Étape 3 :

1. Calculer currentLocalOffsetActive :

if ($t_{codeword} \geq \text{timeOfNextJump}$) *and* (*timeOfNextJump* is not 0) *then*

$$\text{currentLocalOffsetActive} = \text{currentLocalOffset} + \text{jumpSeconds}$$

else

$$\text{currentLocalOffsetActive} = \text{currentLocalOffset}$$

endif

2. Calculer le temps du Daily Jam dans le temps PTP :

$$t_{jamPTP} = \text{timeOfNextJam}$$

3. Calculez le nombre de mots de code depuis le SMPTE Epoch au moment où juste après Jam time :

$$n_{nextJam} = \text{ceiling}(t_{jamPTP} \times Ff)$$

if Ff == 30/1.001 then

$$CF = 2$$

else if Ff == 25 then

$$CF = 4$$

endif

- a. Calculer le nombre de mots de code jusqu'au début de la séquence du champ de couleur :

$$n_{cf} = ((CF) - (n_{nextJam} \% CF)) \% CF$$

- b. Calculer le nombre ajusté de mots de code jusqu'au début de la séquence du champ de couleur :

$$n_{nextJam} = n_{nextJam} + n_{cf}$$

4. Déterminez si l'événement Daily Jam s'est produit :

if (n_codeword == n_nextJam) then

/ Daily Jam has occurred */*

Calculate exact date, time, and Time Address value of the Daily Jam event using the method in Section 9.4.2.4 ("Step 4")

else

Go to Step 5 (See Section 9.4.2.5).

endif

Étape 4 :

1. Mise à jour du nombre de mots de code depuis le SMPTE Epoch à jam :

$$n_{previousJam} = n_{codeword}$$

2. Mise à jour de l'heure exacte du Daily Jam en heure PTP :

$$t_{epdJamPTP} = \frac{n_{codeword}}{Ff}$$

3. Calculez l'heure exacte du Daily Jam sur l'échelle de temps locale :

$$t_{epdJamLocal} = t_{epdJamPTP} + \text{currentLocalOffsetActive}$$

4. Réglez l'adresse temporelle à l'heure du Daily Jam :

$$HH_{pdjam} = \text{floor}\left(\frac{t_{epdjamLocal}}{60 \times 60}\right) \% 24$$

$$MM_{pdjam} = \text{floor}\left(\frac{t_{epdjamLocal}}{60}\right) \% 60$$

$$SS_{pdjam} = 0$$

if($Ff == 25$) *and* (*colorFrameIdentificationMode* == 1) *then*

$$FF_{pdjam} = 1$$

else

$$FF_{pdjam} = 0$$

endif

Étape 5 :

1. Calculer le nombre d'images depuis le précédent événement Daily Jam :

$$f_e = n_{codeword} - n_{previousJam}$$

2. Calculez la valeur temporelle de l'adresse temporelle SMPTE ST 12-1 en ajoutant le nombre de trames depuis la précédente, à la valeur de l'adresse temporelle lors du précédent événement Daily Jam :

Voir doc st-2059 dépend des fps

3. Calculer la valeur de la date SMPTE ST 309 : voir doc st-2059

$$t_{elocal} = (t_{epdjamLocal} + \left(\text{floor}\left(\frac{HH_{notmod24}}{24}\right) \times 24 \times 60 \times 60\right))$$

Avec

Étape 6 :

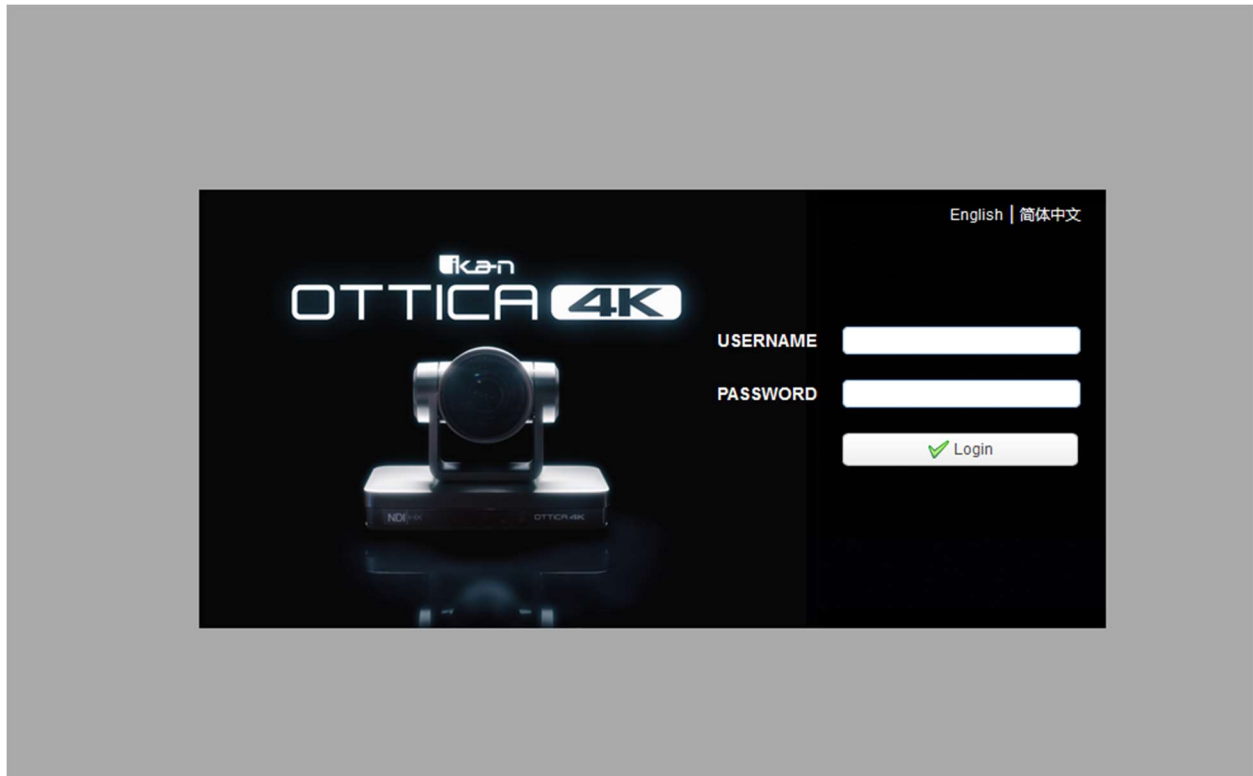
1. Incrémenter le mot-code ; retourner à l'étape 3

$$n_{codeword} = n_{codeword} + 1$$

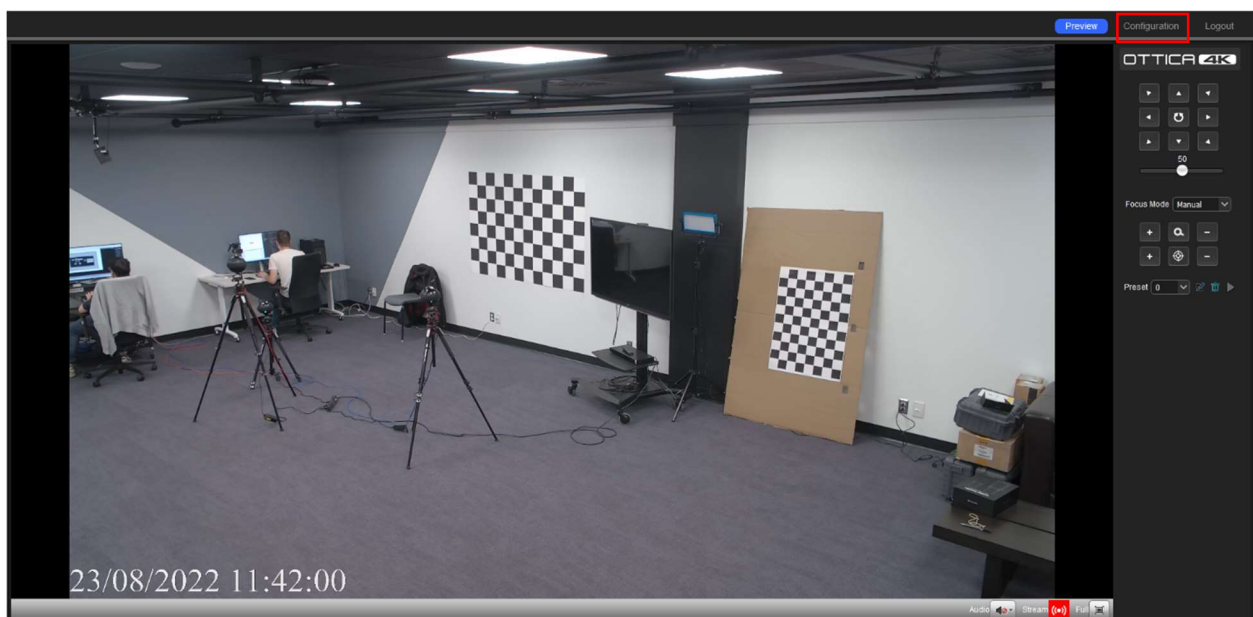
$$t_{codeword} = \frac{n_{codeword}}{Ff}$$

Fonctionnement site web caméra Ottica 4k

Pour se connecter à la caméra Ottica, dans un premier temps il faut aller sur un navigateur internet et mettre dans la barre d'URL : 10.180.137.50, soit l'adresse IP de la caméra que nous souhaitons contrôler.



On arrive ensuite sur cette page. Ou il faut se connecter. Une fois cela fait on arrive ici :



Une fois arrivé là, il faut configurer la caméra, cliquez sur configuration (le rectangle rouge sur l'image au-dessus)

The screenshot displays a configuration interface with a dark theme. On the left is a sidebar menu with three main sections: 'Video Configuration', 'Network Configure', and 'System Configure'. The 'Video Configuration' section is expanded, showing sub-items like 'Video Encoding' (highlighted in blue), 'Stream Publish', 'RTP Multicast', 'Video Parameters', 'Video OSD', 'OSD Font Size', and 'Video Out'. The main area on the right is divided into two columns for comparing two configurations. The top section, 'Video Encoding', includes settings for Compression Format (H.264), Profile (BP vs HP), Video Resolution (3840*2160 vs 320*180), Rate Control (CBR), Image Quality (Best vs Better), Bit Rate (kps) (6000 vs 512), Frame Rate (fps) (25), I Frame Interval (100), I Frame Min QP (20), and Stream Name (live/av0 vs live/av1). A 'Save' button is located at the bottom right of the main area.

Setting	Configuration 1	Configuration 2
Compression Format	H.264	H.264
Profile	BP	HP
Video Resolution	3840*2160	320*180
Rate Control	CBR	CBR
Image Quality	Best	Better
Bit Rate (kps)	6000	512
Frame Rate (fps)	25	25
I Frame Interval	100	100
I Frame Min QP	20	20
Stream Name	live/av0	live/av1

Save

Dans video encding, il est important pour que la synchronisation puisse s'effectuer de bien mettre le Rate control en CBR, qui correspond au débit binaire constant, une technique de contrôle qui maintient le débit binaire constant, mais permet à la qualité vidéo de varier. Dans son mode VBR, j'ai constaté une dérive due à la variabilité du débit.

4 Video Configuration

- Video Encoding
- Stream Publish**
- RTP Multicast
- Video Parameters
- Video OSD
- OSD Font Size
- Video Out

4 Network Configure

- Network Port
- Ethernet

Enable ☒

Protocol Type RTMP

Host Address 10.180.137.50

Host Port 1935

Stream Name live/av0

Username

Password

Save

Stream Publish permet de gérer tout ce qui est publication du stream, ainsi à partir de ces informations on peut récupérer l'adresse du flux qui est dans ce cas : 'rtmp://10.180.137.50:1935/live/av0' ou de façon plus générique dans le cas d'un flux rtmp : **rtmp://{host address} :{host port}/{stream name}**

Enfin le dernier paramètre intéressant dans le cas de la configuration d'un flux est le paramètre Video Out qui sert à définir la qualité et le framerate du flux

Configurations

- Audio Configuration
- 4 **Video Configuration**
 - Video Encoding
 - Stream Publish
 - RTP Multicast
 - Video Parameters
 - Video OSD
 - OSD Font Size
 - Video Out**

Video Out

Video Out Format 3840x2160P25

Save

Fonctionnement API Insta360 :

L'API de caméra Insta360 Pro définit un ensemble de commandes pour contrôler les caméras de la série Insta360 Pro, dont l'Insta360 Pro 2. Les commandes sont transférées via requête/réponse http, les données sont encodées au format JSON. La caméra prenant en charge l'API de caméra Insta360 Pro dispose d'un serveur http intégré. Vous pouvez obtenir l'adresse IP à partir de l'écran de la caméra. Voici les routeurs d'URL.

Name	URL Pattern	Function
<code>command</code>	<code>http://{camera_ip}:20000/osc/commands/execute</code>	Execute commands
<code>state</code>	<code>http://{camera_ip}:20000/osc/state</code>	Poll state, also for notification from camera to client
<code>file</code>	<code>http://{camera_ip}:8000/{fileuri}</code>	Access Image/Video file from camera
<code>preview</code>	<code>rtmp://{camera_ip}:1935/live/preview</code>	rtmp stream of preview

Dans notre cas, les commandes les plus importantes sont `command` et `state`.

Les commandes sont exécutées par des requêtes HTTP POST. Les en-têtes HTTP requis sont répertoriés ci-dessous.

Name	Value	Description
Fingerprint	A string value, empty string for <code>connect</code> command	Identification of the client, obtained by <code>connect</code> command
Content-Type	application/json	

La plupart des commandes ont une structure similaire du corps de la requête. L'objet JSON racine a des attributs de nom et de paramètres, le nom décrit la commande à exécuter. les paramètres diffèrent entre les différentes commandes.

Généralement, la réponse contient le nom et l'état. `state` indique si la requête a été exécutée avec succès. La valeur est effectuée en cas de succès, d'exception ou d'erreur dans le cas contraire. Par exemple pour une tentative de connexion à une caméra :

On success

```
{
  "name": "camera._connect",
  "state": "done",
  "results": { //results contains the data returned by the command.
    ...
  }
}
```

On failure

```
{
  "name": "camera._connect",
  "state": "exception",
  "error": { //error object contains the detailed error
    "code": -1, //error code
    "description": "The camera has been taken by another client." //error description
  }
}
```

Maintenant parlons des différentes commandes que nous allons utiliser dans notre cas.

connect est la première commande à exécuter. En cas de succès, une session entre la caméra et le client est établie, au cours de laquelle aucun autre client ne peut se connecter à cette caméra. Une fois la connexion établie, le résultat nous donne un "Fingerprint": "BL-xZiXM7rk=", par exemple qui sert de clé pour communiquer avec la caméra en question.

Start live permet de lancer un live. La caméra de la série Insta360 Pro dispose de 2 modes en direct, normal et d'origine en direct. Pour le mode normal, la caméra envoie un seul flux équivalent rectangulaire. C'est le mode en direct le plus couramment utilisé, mais avec une résolution maximale de 3840x3840 en raison de la limite d'assemblage intégrée. Pour le mode direct d'origine, la caméra envoie plusieurs flux fish-eye (un pour chaque objectif). Ce mode est principalement utilisé pour l'assemblage hors caméra, et la résolution va jusqu'à 8K (nécessite que le matériel client soit capable de faire l'assemblage en temps réel). Pour le mode direct d'origine, si la liveUrl que vous avez spécifiée est rtmp://{host}/{app_name}, vous pouvez accéder aux flux par l'URL rtmp://{host}/{app_name}/origin{i} où i= 1,2,3,4,5,6

```
{
  "name": "camera._startLive",
  "parameters":{
    "origin":{
      "mime":string, string, //mime type of the output, might be "h264","h265"
      for video, "jpeg","raw" for image,
      "width":int,
      "height":int,
      "framerate":float, //bitrate in Kbps
      "bitrate":int,
      "logMode":int,
      "liveUrl":string, //only available in origin live mode, pass rtmp url
      without stream name. eg. rtmp://127.0.0.1/live
      "saveOrigin":bool //whether to save the separated fisheye origin
      video/image.

    },

    "stitching":{ //stitching is only needed in normal mode, do not pass this
      param for origin live mode
      "mode":string, //stitching mode, "pano" for mono 360 video, "3d_top_left"
      for stereo 360 with top/bottom layout, left eye on top. "3d_top_right" for
      stereo 360 with top/bottom layout, right eye on top.
      "mime":string, //mime type of the output, "h264" for video, "jpeg" for
      image
      "width":int,
      "height":int,
      "framerate":float,
      "bitrate":float //bitrate in Kbps
      "map":string,
      "_liveUrl":string, //rtmp url, like rtmp://127.0.0.1/live/test
      "liveOnHdmi":bool,
      "fileSave":bool //save live stream on the camera.
    },
  },
  "audio":{
    "mime":string,
    "sampleFormat":string,
```

```

        "channelLayout":string,
        "samplerate":int,
        "bitrate":int
    }
},
"autoConnect":{
    "enable":bool,
    "interval":int, // retry delay in ms.
    "count":int //count = -1 means always try to reconnect
}
"stabilization":bool
}

```

Ci-dessus une petite explication des paramètres les plus importants.

Pour ce qui est des types de Stream possibles se référer au tableau ci-dessous :

Single Lens Resolution	Max Output Resolution	Support Real-time Stitching	Support 3D Output	Support HDR	Single Lens Bitrate
4000x3000 (Photo Only)	8000x4000 8000x8000(3D)	✓	✓	X	-
3840x2880@30fps	7680x3840 7680x7680(3D)	X	✓	✓	1~120Mbps
3840x2160@30fps	7680x3840	✓	X	X	1~120Mbps
3840x2160@60fps	7680x3840	X	X	X	1~120Mbps
3840x1920@60fps	7680x3840	X	X	X	1~120Mbps
3200x2400@30fps	6400x3200 6400x6400(3D)	✓	✓	X	1~120Mbps
1920x1440@30fps	3840x1920 3840x3840(3D)	✓	✓	X	1~120Mbps
1920x1440@120fps	3840x1920 3840x3840(3D)	X	✓	X	1~120Mbps

La commande `stop_live` arrête le flux en direct et met à jour l'état de la caméra.

L'interrogation d'état est utilisée à la fois pour le rythme cardiaque et pour recevoir une notification de la caméra.

La caméra déconnectera le client après 10 secondes sans recevoir de commande d'état. Généralement, nous suggérons d'envoyer la commande d'état toutes les 1 seconde, n'envoyez pas la commande d'état suivante avant que la précédente ne réponde.

À l'exception des en-têtes requis, la commande d'état n'a besoin d'aucun paramètre.

Maintenant on va voir, l'évolution du code jusqu'au code final :

Solution retenue

La solution retenue dans le cadre de la synchronisation de deux caméras IP est une solution qui consiste en l'ajout d'un timecode sur chaque frames reçu de deux flux vidéo rtmp. A partir de cela, à chaque frame reçu incrémentera le timecode. Au final, on compare les timecodes de chaque frame et si une des frames d'un flux et en avance sur une image de l'autre flux, alors dans ce cas, il suffit de faire patienter l'affichage de la frame en question, le temps que l'autre flux est eu le temps de rattraper son retard.

Explication et évolution de la solution

Etape 1 : Lancement des deux streams

Premièrement, on commence par lancer les deux flux vidéo de chaque caméra. Pour cela, il faut aller dans les settings, se mettre en DHCP, pour que le réseau puisse nous attribuer une adresse IP, n'oublions pas non plus de connecter en Ethernet la caméra au même réseau que l'ordinateur. Ensuite, il suffit de récupérer l'adresse IP directement sur le haut de l'écran de la caméra.



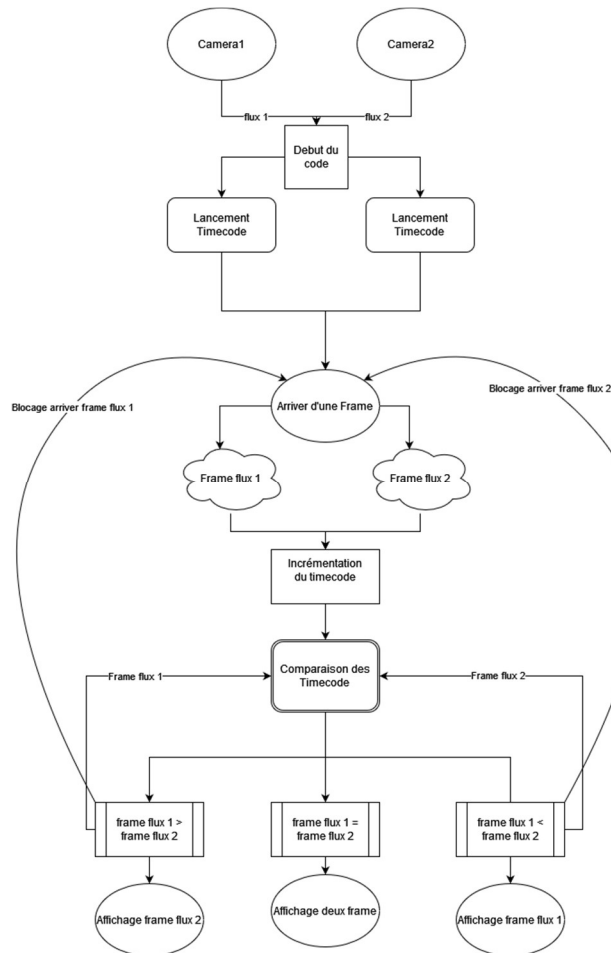
Une fois, cette adresse IP en mémoire, on peut vérifier que la caméra est bien sur le réseau en utilisant la commande « ping {IP_caméra} » dans un terminal. Si la caméra répond au ping alors on peut passer à l'étape suivante.

Il faut lancer l'application "Insta360 pro" renseignés l'IP de la caméra que l'on souhaite utiliser. Puis aller dans l'onglet Live-stream choisir RTMP comme serveur. A partir de là, il faut se déconnecter de la caméra avec l'application mais tout en gardant le live d'actif. Il faut ensuite répéter les opérations vues avec l'autre caméra.

Etape 2 : Code de synchronisation

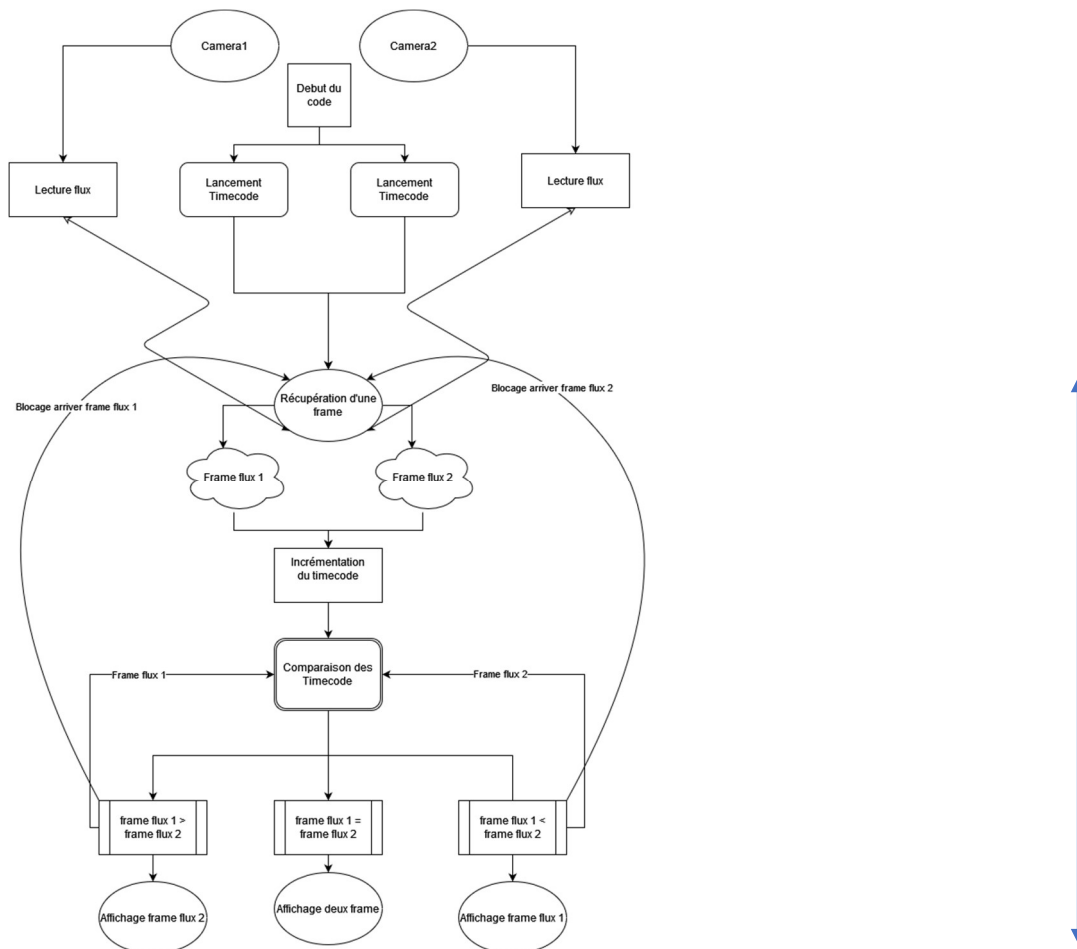
Dans cette étape, on verra quelle méthode de synchronisation j'utilise ainsi que l'explication du code utilisé en lien avec le protocole STMPE.

Schéma explicatif du code



Le problème de ce code est que quand les frames de chaque flux arrivent elles sont stockées dans le buffer d'OpenCV or ce buffer n'a pas une taille infinie. Ce qui au fur et à mesure que le code se déroule pour la première fois les arrivées de flux sont décalé ce qui implique que quand on leur applique un timecode, les timecodes commence en même temps mais à des frames différents. Les différents essais que j'ai pu faire montre que pour deux streams simultanés les variations de frames entre deux flux peuvent aller de 30 frames à 60 frames. Pour résoudre ce problème une solution simple est de faire commencer le timecode avec x frames d'avance au flux en avance. Néanmoins cette méthode s'avère fastidieuse avec plus de 2 streams en simultané. Ainsi, ceci nous amène à aller tester une autre solution le « multiprocessing »

Schéma explicatif du code



Cette méthode où la lecture des flux se fait en parallèle du code central de comparaison des timecodes permet bien de récupérer les premières frames de chaque flux sans décalage. Mais dans ce cas précis un autre problème est de mise en effet comme la lecture des flux vidéo se fait en annexe du code on se retrouve avec une lecture du flux durant la partie de comparaison (flèche bleue) et donc on se retrouve avec un timecode qui n'a plus aucun sens. En effet, chaque thread a le temps de lire au moins 5 frames. Une méthode pour résoudre ce problème consiste à seulement rajouter l'incrément du timecode dans les threads.

En faisant ceci on se retrouve bien avec des timecodes corrects. Et donc l'algorithme de comparaison peut de nouveau fonctionner cependant, le blocage de la lecture de frame n'est plus effectif, en effet les frames étant lu en dehors du code cela implique qu'un blocage de celle-ci ne s'applique que pour les frames qui sont comparés dans le code. On n'a donc des « sauts » d'images. Les flux en avance un fois synchronisée se retrouve donc à réafficher des images beaucoup plus loin dans le temps.

Explication du code de synchronisation des caméras

1/ Méthode

La méthode choisie est un mixe entre deux méthodes de synchronisation la synchronisation par timecode et la synchronisation par analyse des variations lumineuses. Cette solution procède à un alignement des séquences en fonction de leur variation de luminosité. Synchroniser les caméras de cette façon permet de pouvoir contrôler le moment où elles se synchronisent, c'est-à-dire que contrairement à une analyse de mouvement qui peut ne pas être perçue par toutes les caméras. A cela on ajoute des timecodes sur chaque flux de chaque caméra. L'objectif de ce timecode est double. D'un, c'est lui qui permet la synchronisation et de deux il permet de ne pas avoir besoin de faire constamment du traitement d'images pour garder notre synchronisation effective.

2/Matériel

3 caméras insta360 Pro 2 :



1 Ottica 4k :



Toutes ces caméras se trouvent dans une pièce dans laquelle nous sommes en mesure de contrôler la luminosité.



3/ Déroulement et explication d'une synchronisation

Maintenant que les bases le matériel et l'environnement de travail sont posés. On va passer à l'explication du code par un exemple.

Pour se faire nous allons retracer le procédé de synchronisation en analysant chaque partie du code.

1) Connection aux caméras

Le début du code sert à lancer les lives des caméras Insta360 via l'api. Pour cela, on fait des requêtes web via Json aux caméras. Pour pouvoir lancer un live, il faut au préalable se connecter à la caméra.

```
r[i],f[i]=connect(nameIP[i])
print('connection de :', nameIP[i])
jprint(r[i].json())
```

Où connect() est une fonction décrite ci-dessous :

```
def connect(URL):
    command = URL + ":20000/osc/commands/execute"
    post_dic = {}
    post_dic['json'] = {
        "name": "camera._connect",
        "parameters": {
            "hw_time": "MMDDhhmm[[CC]YY][.ss]",
            "time_zone": "GMT+06:00/GMT-06:00"
        }
    }
    response = requests.post(command,**post_dic)

    if response.status_code == 200:
        results = response.json()['results']
        fingerprint = results['Fingerprint']
        results = response.json()
    return response, fingerprint
```

Si la connexion est réussie car la caméra n'est pas prise par un autre utilisateur, alors celle-ci nous renvoie une réponse positive à notre connexion ainsi que « mot de passe » (le fingerprint) qui nous servira pour discuter avec elle par la suite.

A la suite de cela vient la phase de lancement de live. Pour lancer un live via requête on a besoin de savoir quel « post_dic » envoyé. Pour plus d'informations là-dessus, je vous conseille d'aller voir : <https://github.com/Insta360Develop/ProCameraApi> C'est le git de l'api.

```
post_dic[i]['json'] = {
    "name": "camera._startLive",
    "parameters": {
        "origin": {
            "mime": "h265",
            "width": 1920, #to Stream in 4k 1920*1440
            "height": 1440, #2880 for max size,2160 for stitching
            "framerate": 25,
            "bitrate": 20480,
            "liveUrl": nameOrigin[i],
            "saveOrigin": False
        }
    },
}
```

```

        "stiching": {
            "mode": "3d_top_left", # "pano" for mono 360
            video, "3d_top_left" for stereo 360 with top/bottom layout, left eye on top.
            "3d_top_right" for stereo 360 with top/bottom layout, right eye on top.
            "mime": "h264",
            "width": 3840, # 3840*1920 for normal, 7680*3840 for
max #Optimal 1920*1080
            "height": 1920,
            "framerate": 25,
            "bitrate": 10240,
            "_liveUrl": names[i]
        },
        "audio": {
            "mime": 'aac',
            "sampleFormat": 's16',
            "channelLayout": 'stereo',
            "samplerate": 48000,
            "bitrate": 64
        },
        "autoConnect": {
            "enable": True,
            "interval": 5,
            "count": 3
        }
    },
    "stabilization": False,
    "mode": "normal"
}
jprint(execute(post_dic[i], nameIP[i]).json())

```

Cette partie permet de lancer un live avec l'ensemble des paramètres utiles.

```

def execute(post_dic, URL):

    command = URL + ":20000/osc/commands/execute"
    response = requests.post(command, **post_dic)

    if response.status_code == 200:
        connected = True
        results = response.json()

    return response

```

Ceci est la fonction qui permet d'envoyer la commande ci-dessus à la caméra.

Dans le cas de la caméra Ottica 4k on passe par sa page web 10.180.137.50 pour lancer le live.

2) Lancement des threads

Une fois les lives lancés, il faut maintenant accéder à chaque flux avec OpenCV

```
cap = [vStream]*len(names)
# Create a VideoCapture object and read from input file
# If the input is the camera, pass 0 instead of the video file name
for i in range(len(names)):
    cap[i]=vStream(names[i],dispW,dispH)
```

Ceci permet de créer des threads qui fonctionneront en dehors de la boucle principal

Le thread en question :

```
def __init__(self,src,width,height):
    self.width=width
    self.height=height
    self.buffer = queue.Queue(600)
    self.prev_timestamp=-1
    self.timestamps = 0
    self.capture=cv2.VideoCapture(src)
    self.framerate = self.capture.get(cv2.CAP_PROP_FPS)
    self.thread=Thread(target=self.update,args=())
    self.thread.daemon=True
    self.started = True
    self.begin = False
    self.thread.start()

    def update(self):
        while self.started:
            if self.begin == True :
                self.ret,self.frame=self.capture.read()
                self.timestamps = self.capture.get(cv2.CAP_PROP_POS_MSEC)
                if self.timestamps != self.prev_timestamp:
                    self.frame2=cv2.resize(self.frame,(self.width,self.height))
                    self.buffer.put(self.frame2)
                self.prev_timestamp = self.timestamps
                time.sleep(0)
```

Ce thread est utilisé dans le but de remplir un buffer à partir des frames que l'on récupère grâce à la fonction d'OpenCv read(). Ce buffer sera vidé via le billet d'autres fonctions que l'on verra plus tard.

3) Initialisation des timecodes

```
#Timecode creation loop for each stream
for i,c in enumerate(cap):
    if c is not None:
        print(names[i])
        cap[i].start()
        ips = c.getFramerate()
        print(names[i],ips)
        tc[i]=Timecode(ips,'0:0:0.0') #Creation of timecode
```

L'unique but de cette partie est d'initialiser les timecodes de chaque caméra qui nous serviront plus tard pour les synchroniser les unes par rapport aux autres.

4) Détection de variation de lumière et activation des timecodes

Cette partie du code s'oriente autour de la détection d'une variation de luminance, en effet le but est de pouvoir chronométrer la différence de temps entre chaque flux vidéo.

Pour pouvoir faire cela on commence par mettre de côté le frame la plus vieille du buffer et on en profite pour la retirer de celui-ci par l'occasion.

```
# Open the Video
# read the first frame of the video as the initial background image
for i,c in enumerate(cap):
    if c is not None:
        Prev_frame[i]= c.video()
        Prev_frame[i]=cv2.resize(Prev_frame[i], resize_hw)
```

Ou « video() » permet de récupérer la première donnée de mon buffer et de l'enlever de celui par la même occasion.

```
def video(self) :
    return self.buffer.get()
```

Par la suite nous entrons dans une boucle qui ne se finit seulement si l'on a détecté les variations de lumière de chaque flux.

Dans cette boucle la première étape est de récupérer la frame actuelle du buffer soit la frame juste après celle que l'on avait précédemment récupérée.

```

##capture frame by frame

for i,c in enumerate(cap):
    if c is not None:
        Current_frame[i]=cap[i].video()
        Current_frame[i]=cv2.resize(Current_frame[i], resize_hw)

```

A partir de ces deux frames, on commence par les passer en niveau de gris. De ces niveaux de gris on calcule la moyenne de niveau de gris entre 0 et 255 pour chaque pixel de notre frame, pour les frames actuelles et précédentes. A partir de ces deux moyennes, on calcule une différence qui nous servira plus tard.

La dernière partie ne sert qu'à stocker l'ensemble des différences.

```

for i,c in enumerate(cap):
    if c is not None:
        motion[i]=0
        grayPrev[i]= cv2.cvtColor(Prev_frame[i], cv2.COLOR_BGR2GRAY)
        grayCurr[i]= cv2.cvtColor(Current_frame[i], cv2.COLOR_BGR2GRAY)
        tmp = np.average(grayPrev[i])
        tmp2 = np.average(grayCurr[i])

        luminance[i] = tmp2-tmp

        if i == 0 :
            plt1.append(tmp2)
        if i == 1 :
            plt2.append(tmp2)
        if i == 2 :
            plt3.append(tmp2)
        if i == 3:
            plt4.append(tmp2)

```

Par la suite on vérifie que la différence de luminance entre nos deux frames soit supérieure à 80. Ce que l'on considère comme une impulsion lumineuse

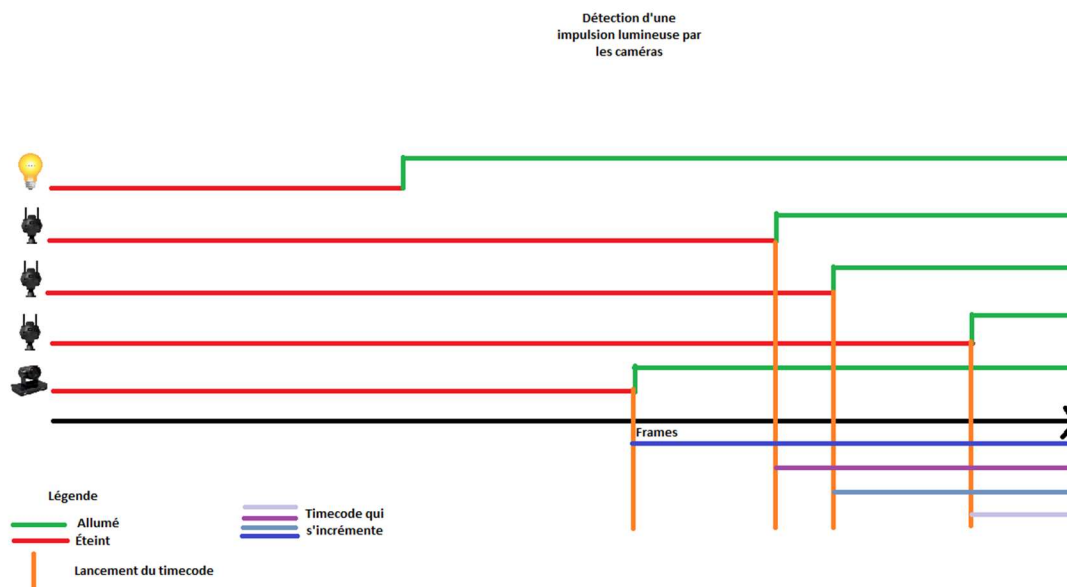
```
# loop to see if the difference is greater than a certain value
for i,c in enumerate(cap):
    if c is not None:
        if luminance[i] > 80 :
            motion[i] = 1
            flaglum[i]=1
for i,c in enumerate(cap):
    if c is not None:
        Prev_frame[i]=Current_frame[i]
```

La variable motion ne sert que pour de l'affichage. Par contre la variable flaglum, elle, sert à lancer les timecode dans la suite du code.

Comme on peut le voir ci-dessous, à partir du moment où l'on a reçu l'impulsion lumineuse, le timecode de la caméra en question est activé et à chaque frame que l'on va recevoir celui-ci s'incrémentera. (tc[i].frames+=1)

```
for i,c in enumerate(cap):
    if c is not None:
        if flaglum[i] :
            tc[i].frames+=1
```

Pour expliquer cela de manière plus simple passons par un schéma :



Le schéma ci-dessus montre une impulsion lumineuse qui se passe à un instant t mais que les caméras reçoivent à un instant t plus un delta de temps différents pour chaque flux. Les timecodes eux, se lancent au moment où l'impulsion apparaît sur les frames.

Ce qui donne si on avait les flux vidéo d'afficher

Avant que l'impulsion lumineuse arrive



L'impulsion lumineuse arrive sur le stream de la caméra Ottica 4k et ainsi le timecode de cette caméra se lance.



Puis successivement, l'ensemble des streams vont voir l'impulsion arrivé et leur timecode vont se lancer.



Une fois que tous les streams ont vu l'impulsion lumineuse par une vérification on peut quitter la boucle.

```
#To break the loop if all of stream sqz the variation of luminance
tmps = 0
for i in range(len(names)):
    if flaglum[i] :
        tmps +=1

if tmps == len(names):
    break
```

Arrivé à ce moment-là, on se retrouve donc avec 4 streams ou chaque frame est relié à un temps.

```
video 0 : 00:00:00.600
video 1 : 00:00:00.600
video 2 : 00:00:00.800
video 3 : 00:00:00.080
```

Comme on peut le voir ci-dessus, juste à la fin de la boucle on trouve 4 streams avec des temporalités différentes, néanmoins on connaît exactement la différence de temps qui sépare chaque stream.

Ainsi à partir de cette différence on est capable grâce à notre buffer de synchroniser les caméras en mettant en pause les streams qui sont avancés par rapport aux autres.

5) Synchronisation temporelle et affichage des streams

Cette partie correspond à la récupération des frames dans le buffer si elles doivent être récupérées. De plus, on continue d'incrémenter le timecode si on a récupéré une frame d'un flux.

```
# Capture frame-by-frame if flag == 1
for i,c in enumerate(cap):
    if c is not None:
        if flag[i] == 1 :
            frames[i] = cap[i].video()
            resize[i]=cv2.resize(frames[i], resize_hw)
            tc[i].frames += 1
```

La boucle ci-dessous a pour utilité de récupérer le timer sous une forme qui nous va bien pour la suite. Ainsi les secondes « pures » deviennent des secondes avec leurs millisecondes.

```
for i,c in enumerate(cap):
    if c is not None:
        h[i],m[i],s[i],ms[i] = frames_to_tc(tc[i], tc[i].frames)
        s[i] = s[i]+ms[i]
    for i in range(len(names)):
        flag[i]=1
```

frames_to_tc() est une fonction dont le code est ci-dessous, son but est d'à partir d'un temps et d'un numéro de frames de récupérer exactement le temps sous la forme d'entier.

```
def frames_to_tc(self, frames):
    """Converts frames back to timecode
    :returns str: the string representation of the current time code
    """
```

```

if self.drop_frame:
    # Number of frames to drop on the minute marks is the nearest
    # integer to 6% of the framerate
    ffps = float(self.framerate)
    drop_frames = int(round(ffps * .066666))
else:
    ffps = float(self._int_framerate)
    drop_frames = 0
# Number of frames per ten minutes
frames_per_10_minutes = int(round(ffps * 60 * 10))
# Number of frames in a day - timecode rolls over after 24 hours
frames_per_24_hours = int(round(ffps * 60 * 60 * 24))
# Number of frames per minute is the round of the framerate * 60 minus
# the number of dropped frames
frames_per_minute = int(round(ffps) * 60) - drop_frames
frame_number = frames - 1
# If frame_number is greater than 24 hrs, next operation will rollover
# clock
frame_number %= frames_per_24_hours
if self.drop_frame:
    d = frame_number // frames_per_10_minutes
    m = frame_number % frames_per_10_minutes
    if m > drop_frames:
        frame_number += (drop_frames * 9 * d) + drop_frames * ((m -
drop_frames) // frames_per_minute)
    else:
        frame_number += drop_frames * 9 * d
ifps = self._int_framerate

frs = frame_number % ifps
if self.fraction_frame:
    frs = round(frs / float(ifps), 3)

secs = int((frame_number // ifps) % 60)
mins = int(((frame_number // ifps) // 60) % 60)
hrs = int((((frame_number // ifps) // 60) // 60))

return hrs, mins, secs, frs

```

Une fois les timers de chaque flux récupéré, nous allons pouvoir les comparer dans le but d'arrêter l'affichage des streams qui sont en avance par rapport aux autres.

```
#Timecode comparison loop and Display the resulting frame
for i in range(len(names)):
    for j in range(len(names)):
        if not i == j :
            timetimes = time_comp(h[i],m[i],s[i],h[j],m[j],s[j])
            if timetimes == 1 :
                print('<')
                if flag[j]==1:
                    flag[j] = 0
            elif timetimes == 0 :
                print('>')
                if flag[i]==1:
                    flag[i]=0
            elif timetimes :
                print("=")
```

La fonction time_comp() est chargé de comparer des temps.

```
def time_comp(heure, minute, seconde, heure2, minute2, seconde2):
    #t1 > t2 -> 0
    #t1 < t2 -> 1
    #t1 = t2 -> 2
    dt = 0.033
    if(heure > heure2):
        return 0
    elif(heure < heure2):
        return 1
    elif(heure == heure2):
        if(minute > minute2):
            return 0
        elif(minute < minute2):
            return 1
        elif(minute == minute2):
            if(seconde > seconde2 and abs(seconde - seconde2) > dt ):
                return 0
            elif(seconde < seconde2 and abs(seconde - seconde2) > dt ):
                return 1
            elif((seconde == seconde2) or abs(seconde - seconde2)<dt):
                return 2
```

Ainsi avec la boucle for on se charge de comparer tous les cas possibles. Par exemple, prenons 3 temps

T1 T2 T3 avec $T1 < T2 < T3$

Dans ce cas-là, la logique du code veut que T1 continue d'avancer et on stop T2 et T3 car ils sont en avance sur T1. Or sans les conditions que l'on trouve avec « flag[i] » on se retrouverait à la fin avec :

T1 continue d'avancer car il est inférieur à T2 et T3

T2 continue d'avancer car il est en retard sur T3

T3 s'arrête le temps que T1 et T2 le rattrape.

Or ce n'est pas ce qui est voulu on cherche à avoir T1 qui rattrape T2. Puis T1 et T2 qui rattrape T3. D'où l'intérêt de la condition.

Ainsi on se retrouve avec :

Premier passage dans la boucle :

video 0 : 00:00:00.600

video 1 : 00:00:00.600

video 2 : 00:00:00.800

video 3 : 00:00:00.080

Second passage:

video 0 : 00:00:00.600

video 1 : 00:00:00.600

video 2 : 00:00:00.800

video 3 : 00:00:00.120

Vidéo 3 va s'incrémenter jusqu'à qu'elle est rattrapé Vidéo 0 et 1 **puis** :

video 0 : 00:00:00.640

video 1 : 00:00:00.640

video 2 : 00:00:00.800

video 3 : 00:00:00.640

Et cela, jusqu'au moment où toutes les vidéos soit synchronisé.

video 0 : 00:00:00.800

video 1 : 00:00:00.800

video 2 : 00:00:00.800

video 3 : 00:00:00.800

Ainsi si les flags[i] sont à 0 alors la lecture de la frame au début de la boucle ne se fait pas. Et donc le Stream est en pause.

Enfin il ne reste plus que l'affiche des frames

```
im_tile = concat_tile([[resize[0], resize[1]],
                      [resize[2], resize[3]]])
cv2.imshow('data/dst/opencv_concat_tile.jpg', im_tile)
```

Pour l'affichage, on concatène les 4 frames de chaque flux en une seule et on l'affiche.

6) Vérification de la synchronisation

Cette partie a pour unique but de vérifier que la synchronisation c'est bien passé. Elle est donc extrêmement facile à expliquer puisque c'est exactement le même fonctionnement que dans la partie 4)

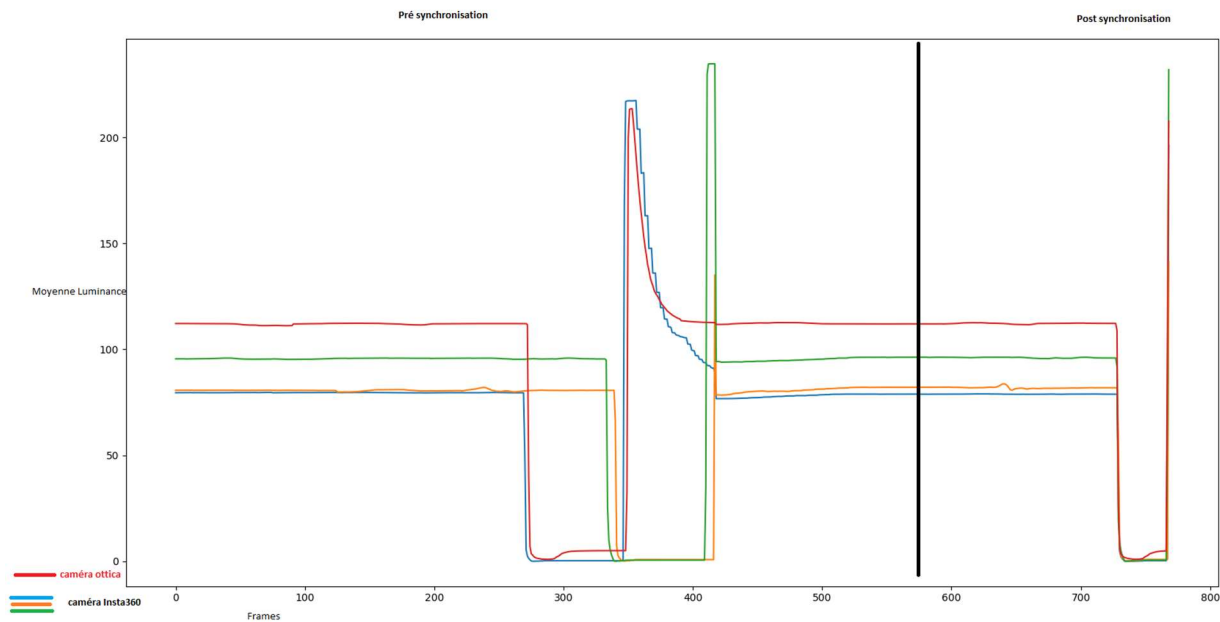
```
flaglum= [0] * len(names)
for i,c in enumerate(cap):
    if c is not None:
        Prev_frame[i]= c.video()
        Prev_frame[i]=cv2.resize(Prev_frame[i], resize_hw)
while True:
    for i,c in enumerate(cap):
        if c is not None:
            Current_frame[i]=cap[i].video()
            Current_frame[i]=cv2.resize(Current_frame[i], resize_hw)
    for i,c in enumerate(cap):
        if c is not None:
            motion[i]=0
            grayPrev[i]= cv2.cvtColor(Prev_frame[i], cv2.COLOR_BGR2GRAY)
            grayCurr[i]= cv2.cvtColor(Current_frame[i], cv2.COLOR_BGR2GRAY)
            tmp = np.average(grayPrev[i])
            tmp2 = np.average(grayCurr[i])
            luminance[i] = tmp2-tmp
            if i == 0 :
                plt1.append(tmp2)
            if i == 1 :
                plt2.append(tmp2)
            if i == 2 :
                plt3.append(tmp2)
            if i == 3:
                plt4.append(tmp2)
    for i,c in enumerate(cap):
        if c is not None:
            if luminance[i] > 80:
                motion[i] = 1
                flaglum[i]=1
    for i,c in enumerate(cap):
        if c is not None:
            Prev_frame[i]=Current_frame[i]
```

```

print(motion)
tmps = 0
for i in range(len(names)):
    if flaglum[i] :
        tmps +=1
if tmps == len(names):
    break
time.sleep(0)

```

Ainsi une fois cela fait, on peut afficher les courbes de différences de luminance en fonction des frames. Comme on peut le voir ci-dessous :



Avant la synchronisation on observe bien que les quatre flux ne sont pas en même temps en effet la chute de luminance qui symbolise une extinction des lumières. On remarque bien que la chute ne se produit pas du tout sur la même frame donc les streams ne sont pas synchronisés. Ainsi après la synchronisation (la partie 5)) on remarque bien cette fois-ci que l'ensemble des streams sont synchronisés. En effet, que ce soit pour la chute de lumière ou l'impulsion On voit que ça arrive sur la même image. Donc on en conclut que notre synchronisation a fonctionné, c'est-à-dire que tout événement qui se produit à un instant t se produira à un instant $t + dt$ pour toutes les caméras.

7) Fermeture des streams

On commence par stopper l'ensemble des Threads que l'on a lancé.

```
# When everything done, release the video capture object
for c in cap:
    if c is not None:
        cap[i].stop()
```

Ou stop() et la fonction d'arrêt.

```
def stop(self) :
    self.started = False
    self.thread.join()
```

Enfin, nous finissons par couper les streams de chaque caméras grâce une commande exécutée en Json comme vu précédemment.

```
#To close all stream of Insta360
for i in range(len(nameIP)):
    r[i],f[i]=connect(nameIP[i])
    print('connection de :', nameIP[i])
    jprint(r[i].json())

    post_dic[i] = {}
    post_dic[i]['headers'] = {}
    post_dic[i]['headers']['content-type'] = 'application/json'
    post_dic[i]['headers']['Fingerprint'] = f[i]
    res = state(f[i], nameIP[i])
    jprint(res.json())
    post_dic[i]['json'] = {"name": "camera._stopLive"}
    jprint(execute(post_dic[i], nameIP[i]).json())
```

4/ Conclusion

En conclusion on a vu le fonctionnement de ce code de synchronisation par timecode et analyse de variation lumineuse. Ce code fonctionne pour des caméras qui ont le même framerate en effet lors de la phase de synchronisation quand la comparaison des timecode à lieu si un des buffers ne se remplit pas à la même vitesse que les autres on va se retrouver avec des situations ou les frames ne seront plus synchronisés et une dérive va apparaître. Cependant, avec les caméras utilisées il est possible de changer le framerate et donc de les mettre toutes à un framerate équivalent. Attention dans le cas des Insta360, un framerate demandé à 30 fps donnera en réalité un framerate à 29.97. Il faut donc vérifier ceci avec une fonction OpenCV «. get(cv2.CAP_PROP_FPS) »

Piste d'évolution :

Le code actuel ne marche que pour des caméras ayant le même frame rate. En effet, la caméra Ottica fonctionnant à un frame rate de 30 images par seconde et les

Insta360 à un frame rate de 29.97 FPS, il ne m'était pas possible de synchroniser les caméras par le biais de ma méthode. En effet, si les frame rates sont différents, on se retrouve avec une dérive qui va au fur et à mesure grandir. C'est à dire que les flux de chaque caméra vont avoir un délai entre eux de plus en plus important. Ainsi, au lieu de changer l'ensemble du code. Par l'api j'ai pu descendre le frame rate des Insta360 à 25 FPS, un frame rate possible pour la Ottica. Et donc synchroniser toutes les caméras.

Cependant j'ai commencé à me pencher sur la question de comment résoudre ces problèmes de frame rate. Voici les quelques pistes d'évolution que je pense être bonne à prendre. D'un le problème de dérive pourrait être dû à mes comparaisons de timecodes qui se ferait mal. En effet, si les trames ne s'affichent pas au moment opportun alors une dérive en découle.

Une autre théorie serait par rapport aux buffers, en effet ceux-ci ne se remplissant pas de la même vitesse, cela pourrait provoquer une dérive. La comparaison des timecode se faisant à un moment x qui est le même pour chaque timecode alors on se retrouvera avec un buffer plus rempli que les autres et donc la comparaison pourrait se faire pour des trames qui sont extrêmement différentes d'une caméra à l'autre due à la dérive.

Enfin la dernière théorie est la plus possible selon moi, serait que la dérive est due à la façon dont les timecodes sont comparés. En effet actuellement comme on peut le voir dans le code, l'ensemble des flux sont comparés les uns par rapport aux autres. Ainsi, l'on n'a aucun flux qui va donner le tempo aux autres. Je m'explique, si les frame rates de chaque caméra sont différents alors le flux final doit forcément avoir le frame rate d'une des caméras or si on compare toutes les caméras entre elles et non par rapport à une seule alors comment fait-on pour connaître le frame rate de notre nouveau flux ? Ainsi le fait que l'on ne se place pas dans un repère d'une caméra en particulier pourrait expliquer la dérive.

Enfin, une autre piste d'évolution qui me paraît importante d'être noté est le fait qu'il est actuellement impossible de Stream avec les caméras en 60 ou 120 fps car le stitching de celles-ci ne peut pas être fait en interne à ces frame rates là. Donc, pour pouvoir faire un environnement immersif en réalité virtuel, fonctionner à ces frame rates paraît un peu vital. Pour cela, d'après mon analyse, il faudrait pouvoir récupérer les flux origines c'est à dire de chaque caméras fish-eyes au frame rate souhaité et réussir à faire le stitching depuis une machine avec plus de puissance de calcul que les caméras. Pour cela je pense que le mieux serait de passer par trois nano ordinateur tel que des Raspberry pi ou bien des Jetson nano, qui s'occuperaient de récupérer les 6 flux et d'en faire un panorama et de le renvoyer par rtmp comme actuellement. Cela permettrait de bien évidemment réduire grandement la puissance de calcul que si on ne le faisait que sur un seul ordinateur.

8) Annexe

```
import cv2
from timecode import Timecode
from threading import Thread
import numpy as np
import time
import requests
import json
from datetime import datetime
from art import *
from matplotlib import pyplot as plt
import queue
```

<https://docs.opencv.org/4.x/>

<https://pypi.org/project/timecode/>

<https://docs.python.org/3/library/threading.html>

<https://pypi.org/project/requests/>

<https://matplotlib.org/>

<https://docs.python.org/3/library/queue.html>

<https://github.com/Insta360Develop/ProCameraApi>

Code

```
import cv2
from timecode import Timecode
from threading import Thread
import numpy as np
import time
import requests
import json
from datetime import datetime
from art import *
from matplotlib import pyplot as plt
import queue

class vStream:
    def __init__(self,src,width,height):
        self.width=width
        self.height=height
        self.buffer = queue.Queue(600)
        self.prev_timestamp=-1
        self.timestamps = 0
        self.capture=cv2.VideoCapture(src)
        self.framerate = self.capture.get(cv2.CAP_PROP_FPS)
        self.thread=Thread(target=self.update,args=())
        self.thread.daemon=True
        self.started = True
        self.begin = False
        self.thread.start()

    def update(self):
        while self.started:
            if self.begin == True :
                self.ret,self.frame=self.capture.read()
                self.timestamps = self.capture.get(cv2.CAP_PROP_POS_MSEC)
                if self.timestamps != self.prev_timestamp:
                    self.frame2=cv2.resize(self.frame,(self.width,self.height))
                    self.buffer.put(self.frame2)
                self.prev_timestamp = self.timestamps
                time.sleep(0)

    def video(self) :
        return self.buffer.get()

    def getFrame(self):
        return self.ret, self.frame2
```

```

def getFramerate(self):
    return self.framerate

def TMSP(self):
    return self.timestamps

def start(self):
    self.begin = True

def end(self):
    self.begin = False

def getbegin(self):
    return self.begin

def size(self):
    return self.buffer.qsize()

def stop(self) :
    self.started = False
    self.thread.join()

def __exit__(self, exc_type, exc_value, traceback) :
    self.stream.release()

def time_comp(heure, minute, seconde, heure2, minute2, seconde2):
    #t1 > t2 -> 0
    #t1 < t2 -> 1
    #t1 = t2 -> 2
    dt = 0.033
    if(heure > heure2):
        return 0
    elif(heure < heure2):
        return 1
    elif(heure == heure2):
        if(minute > minute2):
            return 0
        elif(minute < minute2):
            return 1
        elif(minute == minute2):
            if(seconde > seconde2 and abs(seconde - seconde2) > dt ):
                return 0
            elif(seconde < seconde2 and abs(seconde - seconde2) > dt ):
                return 1
            elif((seconde == seconde2) or abs(seconde - seconde2)<dt):

```

```

        return 2

def frames_to_tc(self, frames):
    """Converts frames back to timecode
    :returns str: the string representation of the current time code
    """
    if self.drop_frame:
        # Number of frames to drop on the minute marks is the nearest
        # integer to 6% of the framerate
        ffps = float(self.framerate)
        drop_frames = int(round(ffps * .066666))
    else:
        ffps = float(self._int_framerate)
        drop_frames = 0

    # Number of frames per ten minutes
    frames_per_10_minutes = int(round(ffps * 60 * 10))

    # Number of frames in a day - timecode rolls over after 24 hours
    frames_per_24_hours = int(round(ffps * 60 * 60 * 24))

    # Number of frames per minute is the round of the framerate * 60 minus
    # the number of dropped frames
    frames_per_minute = int(round(ffps) * 60) - drop_frames

    frame_number = frames - 1

    # If frame_number is greater than 24 hrs, next operation will rollover
    # clock
    frame_number %= frames_per_24_hours

    if self.drop_frame:
        d = frame_number // frames_per_10_minutes
        m = frame_number % frames_per_10_minutes
        if m > drop_frames:
            frame_number += (drop_frames * 9 * d) + drop_frames * ((m -
drop_frames) // frames_per_minute)
        else:
            frame_number += drop_frames * 9 * d

    ifps = self._int_framerate

    frs = frame_number % ifps
    if self.fraction_frame:
        frs = round(frs / float(ifps), 3)

```

```

        secs = int((frame_number // ifps) % 60)
        mins = int(((frame_number // ifps) // 60) % 60)
        hrs = int((((frame_number // ifps) // 60) // 60))

        return hrs, mins, secs, frs

def date():

    year = datetime.now().year
    month = datetime.now().month
    day = datetime.now().day
    hour = datetime.now().hour
    minute = datetime.now().minute
    second = datetime.now().second
    year = str(year)
    if month < 10 :
        month = '0'+str(month)
    else :
        month = str(month)
    if day < 10 :
        day = '0'+str(day)
    else :
        day = str(day)
    if hour < 10 :
        hour = '0'+str(hour)
    else :
        hour = str(hour)
    if minute < 10 :
        minute = '0'+str(minute)
    else :
        minute = str(minute)
    if second-1 < 10 :
        second = '0'+str(second-1)
    else :
        second = str(second)

    return month+day+hour+minute+year+second

def jprint(obj):
    # create a formatted string of the Python JSON object
    text = json.dumps(obj, sort_keys=True, indent=4)
    print(text)

def connect(URL):

```

```

command = URL + ":20000/osc/commands/execute"
post_dic = {}
post_dic['json'] = {
    "name": "camera._connect",
    "parameters":
        {
            "hw_time": "MMDDhhmm[[CC]YY][.ss]",
            "time_zone": "GMT+08:00/GMT-08:00"
        }
}
response = requests.post(command,**post_dic)

if response.status_code == 200:
    results = response.json()['results']
    fingerprint = results['Fingerprint']
    results = response.json()
    return response, fingerprint

def execute(post_dic, URL):

    command = URL + ":20000/osc/commands/execute"
    response = requests.post(command,**post_dic)

    if response.status_code == 200:
        connected = True
        results = response.json()

    return response

def state(Fingerprint, URL):

    command = URL + ":20000/osc/state"
    post_dic = {}
    post_dic['headers'] = {}
    post_dic['headers']['content-type'] = 'application/json'
    post_dic['headers']['Fingerprint'] = Fingerprint
    response=requests.post(command,**post_dic)
    return response

def concat_tile(im_list_2d):
    return cv2.vconcat([cv2.hconcat(im_list_h) for im_list_h in im_list_2d])

#=====
=====
#                                     variable
#
#

```

```

#=====
=====

#variable of task request for API Insta360
post_dic = {}
post_dic['headers'] = {}
post_dic['headers']['content-type'] = 'application/json'

#URL of Insta360
#-----
URL1 = "http://10.180.137.156"
URL2 = "http://10.180.137.158"
URL3 = "http://10.180.137.160"

#URL of Stream and origin live (live from fisheyes cameras)
#-----
url_rtsp= 'rtsp://10.180.137.51/live/live'
url_rtmp1 = 'rtmp://10.180.137.156/live/live'
url_rtmp2 = 'rtmp://10.180.137.158/live/live'
url_rtmp3 = 'rtmp://10.180.137.160/live/live'
url_rtmp4='rtmp://10.180.137.50:1935/live/av0'
url_hsl = "http://10.180.137.51:8000/tmp/live/live.m3u8"

url_origin1 = 'rtmp://10.180.137.156/live'
url_origin2 = 'rtmp://10.180.137.158/live'
url_origin3 = 'rtmp://10.180.137.160/live'

url_rtmp1_origin1 = 'rtmp://10.180.137.156/live/origin1'
url_rtmp1_origin2 = 'rtmp://10.180.137.156/live/origin2'
url_rtmp1_origin3 = 'rtmp://10.180.137.156/live/origin3'
url_rtmp1_origin4 = 'rtmp://10.180.137.156/live/origin4'
url_rtmp1_origin5 = 'rtmp://10.180.137.156/live/origin5'
url_rtmp1_origin6 = 'rtmp://10.180.137.156/live/origin6'

url_rtmp2_origin1 = 'rtmp://10.180.137.158/live/origin1'
url_rtmp2_origin2 = 'rtmp://10.180.137.158/live/origin2'
url_rtmp2_origin3 = 'rtmp://10.180.137.158/live/origin3'
url_rtmp2_origin4 = 'rtmp://10.180.137.158/live/origin4'
url_rtmp2_origin5 = 'rtmp://10.180.137.158/live/origin5'
url_rtmp2_origin6 = 'rtmp://10.180.137.158/live/origin6'

url_rtmp3_origin1 = 'rtmp://10.180.137.160/live/origin1'
url_rtmp3_origin2 = 'rtmp://10.180.137.160/live/origin2'
url_rtmp3_origin3 = 'rtmp://10.180.137.160/live/origin3'
url_rtmp3_origin4 = 'rtmp://10.180.137.160/live/origin4'

```

```

url_rtmp3_origin5 = 'rtmp://10.180.137.160/live/origin5'
url_rtmp3_origin6 = 'rtmp://10.180.137.160/live/origin6'
#-----

#Creation of stream array
nameIP = [URL1 , URL2, URL3]
nameOrigin = [url_origin1 , url_origin2, url_origin3]
names = [url_rtmp1, url_rtmp2, url_rtmp3, url_rtmp4]
window_titles = ['first',
'second','1','2','3','4','5','6','7','8','9','10','11','12','13','14','15','16','17','18','19','20','21','22','23','24','25','26','27','28','29','30','31','32','33','34','35','36','37','38','39','40','41','42','43','44','45','46']
r = [None]*len(names)
f = [None]*len(names)
post_dic = [None]*len(names)

#Variable
frames = [None] * len(names) #Frame
ret = [None] * len(names) #Bool
resize = [None] * len(names) #To resize windows
tc = [None] * len(names) #timecode
h = [None] * len(names) #hours
m = [None] * len(names) #minutes
s = [None] * len(names) #seconds
ms = [None] * len(names) #milliseconds
flag = [1] * len(names) #flag to block the read of a frame 1 -> frame pass
0-> block

resize_hw = (960,480) #Size of resize windows

brightflag=0 #Flag
Prev_frame = [None] * len(names) #Previous Frame
Current_frame = [None] * len(names) #Actual Frame
motion = [None] * len(names)
flaglum= [0] * len(names) #Flag if variation of Light
grayPrev = [None] * len(names)
grayCurr = [None] * len(names)
luminance = [None] * len(names)

#Array for graph
plt1 = []
plt2 = []
plt3 = []
plt4 = []

```

```

#Size of frame
dispW=1920
dispH=1080

#buffer mini size

buffer_size = 0

#=====

=====

for i in range(len(nameIP)):
    r[i],f[i]=connect(nameIP[i])
    print('connection de :', nameIP[i])
    jprint(r[i].json())

    print('=====')
=====')

    post_dic[i] = {}
    post_dic[i]['headers'] = {}
    post_dic[i]['headers']['content-type'] = 'application/json'
    post_dic[i]['headers']['Fingerprint'] = f[i]
    res = state(f[i], nameIP[i])
    print('connection de :', nameIP[i])
    jprint(res.json())

    print('=====')
=====')

#Stream Parameters for Insta360 see
https://github.com/Insta360Develop/ProCameraApi for more info
    post_dic[i]['json'] = {
        "name": "camera._startLive",
        "parameters": {
            "origin": {
                "mime": "h265",
                "width": 1920, #to Stream in 4k 1920*1440
                "height": 1440, #2880 for max size,2160 for stitching
                "framerate": 25,
                "bitrate": 20480,
                "liveUrl": nameOrigin[i],
                "saveOrigin": False
            }
        }
    }

```

```

        },
        "stiching": {
            "mode": "3d_top_left", # "pano" for mono 360
video,"3d_top_left" for stereo 360 with top/bottom layout, left eye on top.
"3d_top_right" for stereo 360 with top/bottom layout, right eye on top.
            "mime": "h264",
            "width": 3840, # 3840*1920 for normal, 7680*3840 for
max #Optimal 1920*1080
            "height": 1920,
            "framerate": 25,
            "bitrate": 10240,
            "_liveUrl": names[i]
        },
        "audio": {
            "mime": 'aac',
            "sampleFormat": 's16',
            "channellayout": 'stereo',
            "samplerate": 48000,
            "bitrate": 64
        },
        "autoConnect": {
            "enable": True,
            "interval": 5,
            "count": 3
        }
    },
    "stabilization": False,
    "mode": "normal"
}
jprint(execute(post_dic[i], nameIP[i]).json())

tprint('SLEEP')
time.sleep(2)

print('=====')

tprint('Live ON')
cap = [vStream]*len(names)
# Create a VideoCapture object and read from input file
# If the input is the camera, pass 0 instead of the video file name
for i in range(len(names)):
    cap[i]=vStream(names[i],dispW,dispH)

#Timecode creation loop for each stream

```

```

for i,c in enumerate(cap):
    if c is not None:
        print(names[i])
        cap[i].start()
        ips = c.getFramerate()
        print(names[i],ips)
        tc[i]=Timecode(ips,'0:0:0.0') #Creation of timecode

tprint('sleep')

time.sleep(2)

#Sync Lumiere
#-----

print('sync cam pls turn the light On/Off')

# Open the Video
# read the first frame of the video as the initial background image
for i,c in enumerate(cap):
    if c is not None:
        Prev_frame[i]= c.video()
        Prev_frame[i]=cv2.resize(Prev_frame[i], resize_hw)

while True:
    #To fill buffer
    g = 0
    for i,c in enumerate(cap):
        if c.size() > buffer_size:
            g+=1
    if g == len(names):

        ##capture frame by frame
        for i,c in enumerate(cap):
            if c is not None:
                Current_frame[i]=cap[i].video()
                Current_frame[i]=cv2.resize(Current_frame[i], resize_hw)

        # Calculation of the average luminance of each frame to compare it with
        the previous one
        for i,c in enumerate(cap):
            if c is not None:
                motion[i]=0
                grayPrev[i]= cv2.cvtColor(Prev_frame[i], cv2.COLOR_BGR2GRAY)
                grayCurr[i]= cv2.cvtColor(Current_frame[i], cv2.COLOR_BGR2GRAY)

```

```

        tmp = np.average(grayPrev[i])
        tmp2 = np.average(grayCurr[i])

        luminance[i] = tmp2-tmp

        if i == 0 :
            plt1.append(tmp2)
        if i == 1 :
            plt2.append(tmp2)
        if i == 2 :
            plt3.append(tmp2)
        if i == 3:
            plt4.append(tmp2)

# loop to see if the difference is greater than a certain value
for i,c in enumerate(cap):
    if c is not None:
        if luminance[i] > 80 :
            motion[i] = 1
            flaglum[i]=1
        if motion[i] :
            brightflag+=1
    for i,c in enumerate(cap):
        if c is not None:
            Prev_frame[i]=Current_frame[i]

#Start the timecode if he difference is true
print(motion)
for i,c in enumerate(cap):
    if c is not None:
        if flaglum[i] :
            tc[i].frames+=1

#To break the loop if all of stream sqz the variation of luminance
tmpr = 0
for i in range(len(names)):
    if flaglum[i] :
        tmpr +=1

if tmpr == len(names):
    break

time.sleep(0)

# Read until all timecode started

```

```

while True:
    g = 0
    for i,c in enumerate(cap):
        if c.size() > buffer_size:
            g+=1
    if g == len(names):

        # Capture frame-by-frame if flag == 1
        for i,c in enumerate(cap):
            if c is not None:
                if flag[i] == 1 :
                    frames[i] = cap[i].video()
                    resize[i]=cv2.resize(frames[i], resize_hw)
                    tc[i].frames += 1

        #Print actual timecode just for test
        for i,fr in enumerate(frames):
            if fr is not None:
                print('video ',i,':',tc[i])

        #Loop to get the time of timecode
        for i,c in enumerate(cap):
            if c is not None:
                h[i],m[i],s[i],ms[i] = frames_to_tc(tc[i], tc[i].frames)
                s[i] = s[i]+ms[i]
        for i in range(len(names)):
            flag[i]=1

        #Timecode comparison loop and Display the resulting frame
        for i in range(len(names)):
            for j in range(len(names)):
                if not i == j :
                    timetimes = time_comp(h[i],m[i],s[i],h[j],m[j],s[j])
                    if timetimes == 1 :
                        print('<')
                        if flag[j]==1:
                            flag[j] = 0
                    elif timetimes == 0 :
                        print('>')
                        if flag[i]==1:
                            flag[i]=0
                    elif timetimes :
                        print("=")

```

```

        im_tile = concat_tile([[resize[0], resize[1]],
                                [resize[2], resize[3]]])
        cv2.imshow('frame_finale', im_tile)

    # Press Q on keyboard to exit
    if cv2.waitKey(5) & 0xFF == ord('q'):
        break
    time.sleep(0)

# Closes all the frames
cv2.destroyAllWindows()

#Same as Sync Lumiere
#=====
=====
flaglum= [0] * len(names)
for i,c in enumerate(cap):
    if c is not None:
        Prev_frame[i]= c.video()
        Prev_frame[i]=cv2.resize(Prev_frame[i], resize_hw)

while True:
    for i,c in enumerate(cap):
        if c is not None:
            Current_frame[i]=cap[i].video()
            Current_frame[i]=cv2.resize(Current_frame[i], resize_hw)
    for i,c in enumerate(cap):
        if c is not None:
            motion[i]=0
            grayPrev[i]= cv2.cvtColor(Prev_frame[i], cv2.COLOR_BGR2GRAY)
            grayCurr[i]= cv2.cvtColor(Current_frame[i], cv2.COLOR_BGR2GRAY)
            tmp = np.average(grayPrev[i])
            tmp2 = np.average(grayCurr[i])
            luminance[i] = tmp2-tmp
            if i == 0 :
                plt1.append(tmp2)
            if i == 1 :
                plt2.append(tmp2)
            if i == 2 :
                plt3.append(tmp2)
            if i == 3:
                plt4.append(tmp2)
    for i,c in enumerate(cap):
        if c is not None:
            if luminance[i] > 80:

```

```

        motion[i] = 1
        flaglum[i]=1
        if motion[i] :
            brightflag+=1
    for i,c in enumerate(cap):
        if c is not None:
            Prev_frame[i]=Current_frame[i]
    print(motion)
    tmps = 0
    for i in range(len(names)):
        if flaglum[i] :
            tmps +=1
    if tmps == len(names):
        break
    time.sleep(0)

# When everything done, release the video capture object
for c in cap:
    if c is not None:
        cap[i].stop()

#To close all stream of Insta360
for i in range(len(nameIP)):
    r[i],f[i]=connect(nameIP[i])
    print('connection de :', nameIP[i])
    jprint(r[i].json())

    post_dic[i] = {}
    post_dic[i]['headers'] = {}
    post_dic[i]['headers']['content-type'] = 'application/json'
    post_dic[i]['headers']['Fingerprint'] = f[i]
    res = state(f[i], nameIP[i])
    jprint(res.json())
    post_dic[i]['json'] = {"name": "camera._stopLive"}
    jprint(execute(post_dic[i], nameIP[i]).json())

plt.plot(plt1)
plt.plot(plt2)
plt.plot(plt3)
plt.plot(plt4)
plt.ylabel('Luminance')
plt.xlabel('Frames')
plt.show()

```